通用人工智能创新实践课 -LLM API 和 Agent Framework

LLM 应用的技术栈

本节课范围

⑤ 业务应用
会议行动助手

④ Agent 运行时 / 框架
Loop · Tool · State · Memory · Workflow

③ API 协议
Chat Completions · Responses · Messages

② 推理服务
云模型服务 · vLLM · SGLang

① 模型权重
训练后的参数与模型结构

分层职责

⑤ 业务应用	解决具体用户问题
④ Agent 运行时	管理行动、状态与循环
③ API 协议	规范请求与响应格式
② 推理服务	让模型能够被调用
① 模型权重	决定模型会什么

非流式 Chat Completions

一次请求 → 一次完整回答

请求

model： 选择模型　messages： 提交上下文

```
from openai import OpenAI
import os


client = OpenAI(
    base_url=os.environ.get("BASE_URL"),
    api_key=os.environ.get("API_KEY"),
)


completion = client.chat.completions.create(
    model=os.environ.get("MODEL_ID"),
    messages = [
        {"role": "user", "content": "Hello"},
    ],
)
print(completion.choices[0].message.content)
```

响应

choices[0].message.content： 模型回答

```
{
  "id": "chatcmpl-B9MBs8CjcvOU2jLn4n570S5qMJKcT",
  "object": "chat.completion",
  "model": "gpt-5.4",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "Hello! How can I assist you today?",
        "refusal": null,
        "annotations": []
      },
      "logprobs": null,
      "finish_reason": "stop"
    }
  ]
}
```

Chat Completions 中的 Role：从文本输入到对话协议核心论点

概念定义

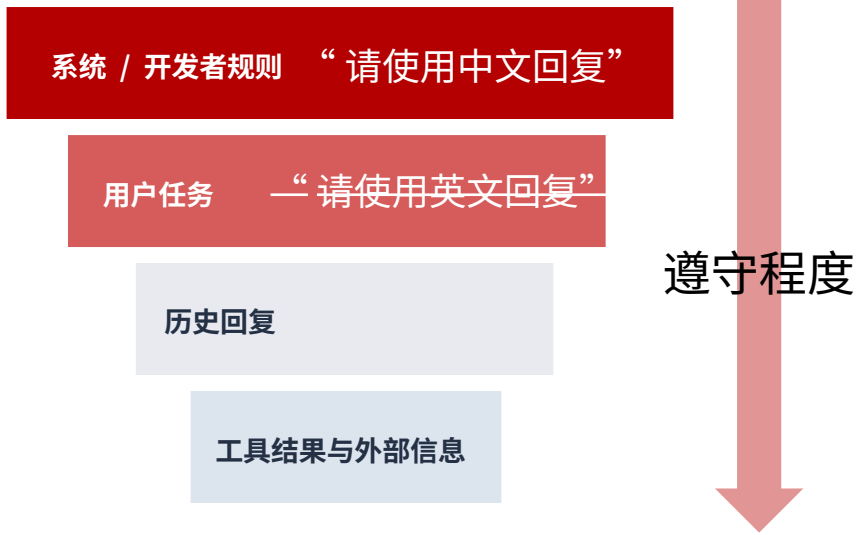
system / developer
高优先级规范：定义任务边界、风格与约束。

user
外部请求：提出问题、目标与补充材料。

assistant
模型历史输出：维护历史记忆。

tool
外部工具结果：供模型继续推理与行动。

```
messages = [
 {"role": "system",
  "content": " 你是严谨的会议纪要助理。 "},
 {"role": "user",
  "content": " 请总结这段会议记录。 "},
 {"role": "assistant",
  "content": " 会议确认下周发布灰度版。 "},
 {"role": "user",
  "content": " 进一步提取待办事项。 "}
]
```



Role 的作用

- 区分消息来源与对话身份
- 建立不同层级的指令关系
- 保持多轮对话的上下文连续性

Role 对模型理解的影响

- 同一段文本，角色不同，语义权重不同
- 上层规则用于定义整体任务与响应原则
- 用户消息负责提出当前任务与具体目标
- 历史回复为后续生成提供语境与状态

Role 对系统设计的意义

- 将自然语言交互转化为结构化消息序列
- 降低规则、输入与历史信息之间的混淆
- 支持工具结果、外部信息等异构内容接入
- 提升多轮交互中的一致性与可控性

流式输出与 SSE




```python
stream = client.chat.completions.create(
    model="<model-id>",
    messages=messages,
    stream=True,
)

for chunk in stream:
    text = chunk.choices[0].delta.content or ""
    print(text, end="", flush=True)
```

核心差异： TTFT （首 token 时间）

流式降低“开始等待”，不减少生成 token 总量；客户端需负责拼接、结束、取消与异常处理。

主流 LLM API 协议

协议	国内模型厂商 支持现状	核心思路
OpenAI Completions	基本不再作为 主流接口	• 把 LLM 当成“文本续写器” • 输入一段 Prompt，输出后续文本
OpenAI Chat Completions	覆盖最广 事实上的通用协议	• 把 LLM 变成“聊天机器人” • 客户端维护会话历史和工具调用 • 服务端无状态 • 历史包袱：在 text-first 基础上不断增加功能
Anthropic Messages	部分支持	• 把聊天升级成“文本 + 图片 + 工具调用” • 把不同能力统一抽象成 Content Blocks，扩展新能力更自然 • 仍由客户端负责会话历史和 Agent 执行循环
OpenAI Responses	部分支持	• “模型 + 工具 + 多步骤执行”的 Agent 接口 • 内置工具 • 可选服务端维护历史会话

核心抽象不断变化

- 早期 API 关注的是：“生成一段文本”
- Chat Completions 关注的是：“生成下一条消息”
- Anthropic Messages 进一步把消息拆成：文本、图片、工具调用等 Content Blocks
- Responses 则开始关注：“完成一次包含模型、工具和多步骤执行的任务”

演化背后的驱动力

- 多轮对话：需要结构化保存角色与上下文
- 多模态：输入输出不再只是字符串
- 工具调用：模型开始产生“动作”，而不仅是文本
- Agent 化：一次模型调用逐步演变为一个持续执行的任务流程

本课程后续默认使用 Chat Completions 协议，但相关概念在不同协议下基本通用

生成参数

参数	影响	常用调法
temperature	输出随机性	事实、抽取、代码任务通常取较低值；创意任务可适当提高。部分推理模型不支持或弱化该参数
max_output_tokens	输出上限	略高于预期最大长度；过低会截断
response_format	输出结构	需要程序稳定解析时优先使用 JSON / JSON Schema ，而不是仅靠 Prompt 约束格式
tool_choice	是否调用工具	auto ： 模型判断； required ： 必须调用；也可指定工具
top_p	采样候选范围	通常与 temperature 选一个调，不同时大幅调整
reasoning_effort	推理深度、延迟和成本	简单任务降低推理强度；复杂推理任务适当提高。仅部分模型支持

生成参数的四类控制

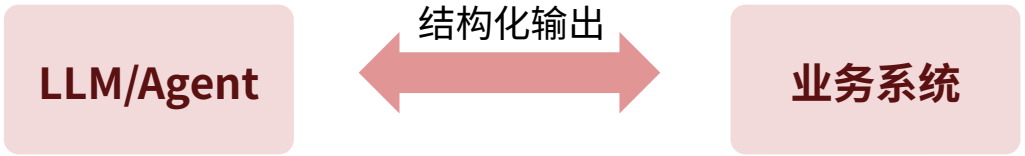
1. 采样控制
   控制模型“如何选择下一个 Token” ，主要影响随机性和多样性。
2. 长度与结构控制
   控制输出“有多长、长什么样”，生产系统中结构化输出通常比单纯调随机性更重要。
3. 工具调用控制
   控制模型“要不要调用工具、调用哪个工具、是否允许并行调用”。
4. 推理控制
   控制模型在复杂任务上投入多少推理资源，通常会同时影响效果、响应时间和成本。

实践原则

- **先使用默认值，再针对具体问题调参**
- 事实、抽取、代码类任务优先保证稳定性
- 需要机器解析时，优先使用 Structured Output / JSON Schema
- Agent 场景重点关注工具定义和 tool_choice
- 推理模型优先关注 reasoning 参数，不要机械套用传统 temperature 经验
- 不同厂商、不同模型的参数名称和支持范围并不完全一致

结构化输出

当模型开始驱动工具、数据库与业务流程，输出就不再只是“文本质量”问题，而是系统间可解析、可校验、可审计的协议问题。



1. **真实业务中的刚需**
   Agent 的输出往往会进入工具调用、工单流转、数据写入、审批触发等链路；格式不稳定会直接变成系统故障或人工返工。
2. **能力从提示词走向协议化**
   实践正在从“让模型照样子写”，演进为由 API 层提供更强约束。
3. **仍需工程兜底**
   不少模型服务只提供有限支持；即便支持 Schema，也只能增强结构可靠性，不能保证事实、权限与业务动作一定正确。

约束逐步增强

① **Prompt + 示例**
只表达格式意图

② **JSON mode**
只保证返回是 JSON，不约束字段

③ **JSON Schema**
约束字段、类型和必填项



json schema

目标 json

多模态输入

模型	Text	Image	Audio	Video
GPT-5.6 Sol				
Gemini 3.x				
Claude Fable 5				
Qwen3.5-Omni				
GLM-5V-Turbo				
MiniMax M3				
Kimi K3				

多模态输入 = 扩展 LLM 的感知能力

- Text： 语言、指令、上下文
- Image： 截图、图表、文档、现实场景
- Audio / Video： 语音、环境、时序信息

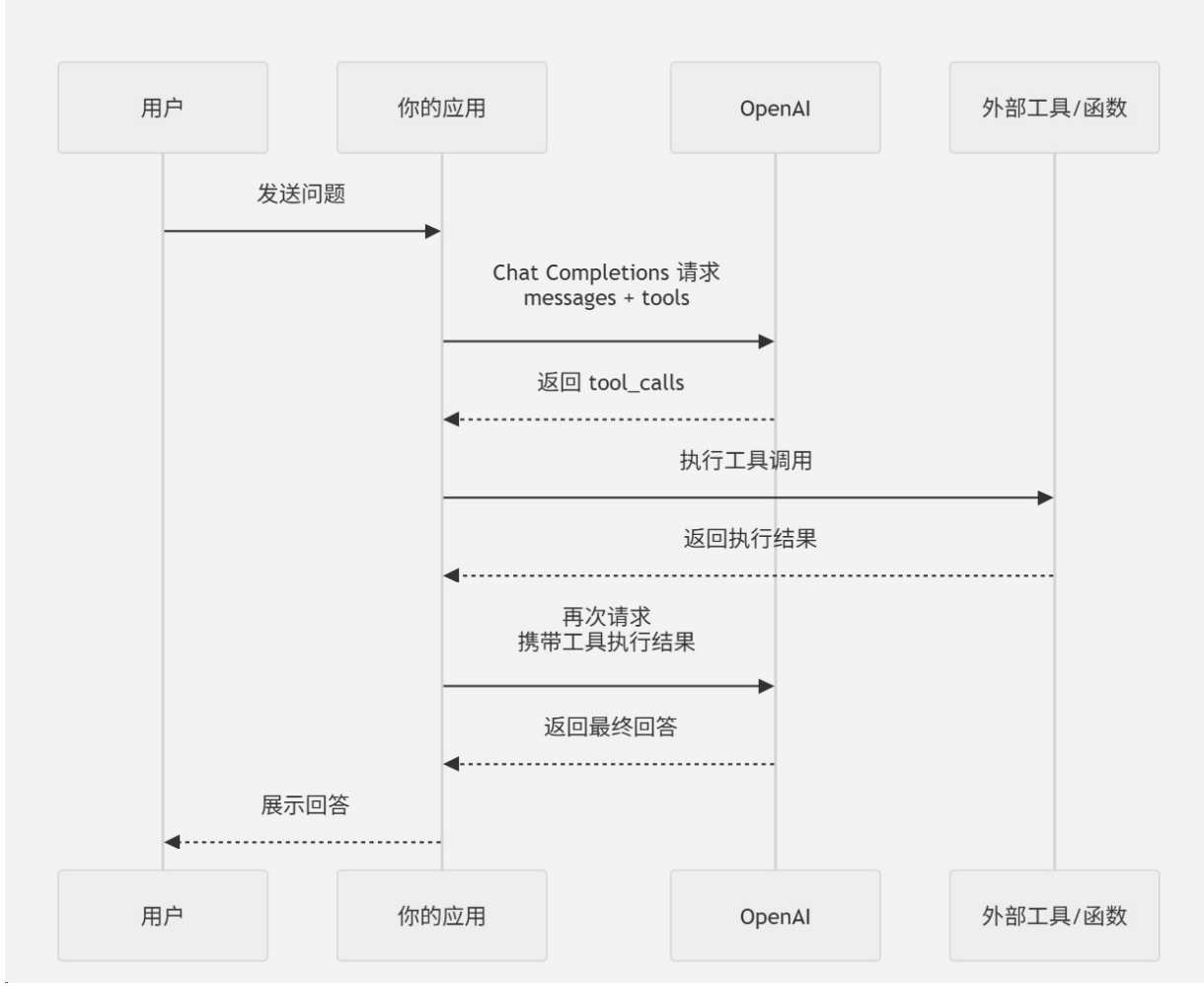
LLM 从“只会读文字” → “能够感知环境”

这也是构建 **Multimodal Agent** 的第一步。

```
messages=[
        {
            "role": "user",
            "content": [
                {"type": "text", "text": "What's in this image?"},
                {
                    "type": "image_url",
                    "image_url": {
                        "url":
"https://upload.wikimedia.org/wikipedia/commons/thumb/d/dd/Gfp-wisconsin-
madison-the-nature-boardwalk.jpg/2560px-Gfp-wisconsin-madison-the-nature-
boardwalk.jpg",
                    }
                },
            ],
        }
    ]
```

方式	适合场景	注意事项
URL	可公开访问的小文件	权限、有效期
base64 / inline	小文件、一次性请求	请求体会变大
file ID	大文件或重复使用	上传、生命周期

工具调用




```
"tools": [
  {
    "type": "function",
    "function": {
      "name": "get_current_weather",
      "description": "Get the current weather in a given location",
      "parameters": {
        "type": "object",
        "properties": {
          "location": {
            "type": "string",
            "description": "The city and state, e.g. San Francisco, CA"
          },
          "unit": {
            "type": "string",
            "enum": ["celsius", "fahrenheit"]
          }
        },
        "required": ["location"]
      }
    }
  }
],
```

工具定义

工具调用（Tool Calling）= 扩展 LLM 的行动能力

- **工具定义**：向模型描述可用工具、功能和参数
- **工具选择**：LLM 根据用户意图判断是否需要调用工具
- **结果回传**：应用执行工具，并将结果返回给 LLM

LLM 从“只会生成内容” → “能够调用外部能力”
可以进一步连接 **搜索、数据库、业务 API 、代码执行等外部系统，**

这也是构建 **AI Agent** 的核心能力之一。

Agent 框架

Agent 框架**位于真实业务与 LLM 之间**。

LLM 负责提出下一步意图，框架真正要解决的是：

- 如何把状态投影成模型上下文；
- 如何将模型输出解析为受约束的动作；
- 如何执行、验证、隔离工具副作用；
- 如何持久化、恢复、重放；
- 如何处理并发、超时、取消、人工审批；
- 如何审计一次运行为什么得到这个结果。

更进一步，如何将一个 Agent 变成一个企业级应用。

当前主流框架已形成三种明显路线

类型	核心	代表
Durable Runtime / 编排内核	图、事件、消息或者状态机	LangGraph 、 LlamaIndex Workflows 、 AutoGen Core 等
Agent Harness / Agent Loop	上下文构造、工具执行	LangChain create_agent Deep Agents smolagents
Control Plane / 企业平台	部署、观测、评测、注册、权限、多租户、审计和成本	LangSmith CrewAI 企业平台 Microsoft Agent Framework 等

Agent 框架共性设计

一个完整 Agent 系统应当分成三平面

控制平面 Control Plane	Registry — Agent / Prompt / Tool / Skill Lifecycle — Deploy / Version / Rollback Governance — IAM / Secret / Policy / Quota Observability — Trace / Eval / Cost / Audit	Agent 如何被定义、发布、治理和运营?
执行平面 Runtime Plane	Orchestrator — Loop / Planner / Routing Context Engine — Assemble / Select / Compact Model Runtime — Gateway / Route / Retry Tool Runtime — Select / Enforce / Execute Durable Execution — Checkpoint / HITL / Resume	一次 Agent Run 如何可靠地完成?
状态与上下文平面 Context & State Plane	Session State — Conversation / Workflow StateMemory — User / Project / Long-term Memory Knowledge — RAG / KG / External Sources Workspace — File / Artifact / CodeRun State — Checkpoint / Event Store / History	Agent 知道什么、记住什么、状态保存在哪里?

Agent 框架共性设计

最小 Agent 循环

```
async def run_agent(state, budget):
    while budget.can_continue():
        context = await context_builder.project(
            state=state,
            memories=await memory.retrieve(state),
            available_tools=await tool_registry.select(state),
        )

        response = await model.generate(context)

        await event_log.append(response)

        if response.is_final:
            return output_validator.validate(response.output)

        intents = tool_call_parser.parse(response)
        decisions = await tool_policy.preflight(intents, state)

        receipts = await tool_executor.execute(decisions)

        state = state_reducer.apply(
            state,
            model_response=response,
            tool_receipts=receipts,
        )

        await checkpoint.save(state)
```

1 2 3 4 5 6

Agent loop：一个可终止的闭环



框架真正的差异在于：

- 循环是否显式；
- State 如何表示；
- 下一步由模型、图、事件还是消息决定；
- 工具调用是否与策略、执行器分离；
- Checkpoint 是否具备完整恢复语义；
- 多 Agent 是 runtime primitive，还是普通工具调用

Agent 框架共性设计

Agent 运行时数据模型

概念	正确定义	生命周期	是否应进入模型上下文
State	当前执行的权威业务状态	单次运行或线程	只投影必要部分
Context	本次模型请求看到的输入视图	单次 LLM 调用	是
Memory	跨轮次、跨运行可检索的信息	长期	检索后按需进入
Artifact	文件、代码、表格、图片、大对象	独立存储	通常只放引用或摘要
Event Log	不可变的运行审计记录	长期	否
Checkpoint	可恢复的状态快照	运行期	否

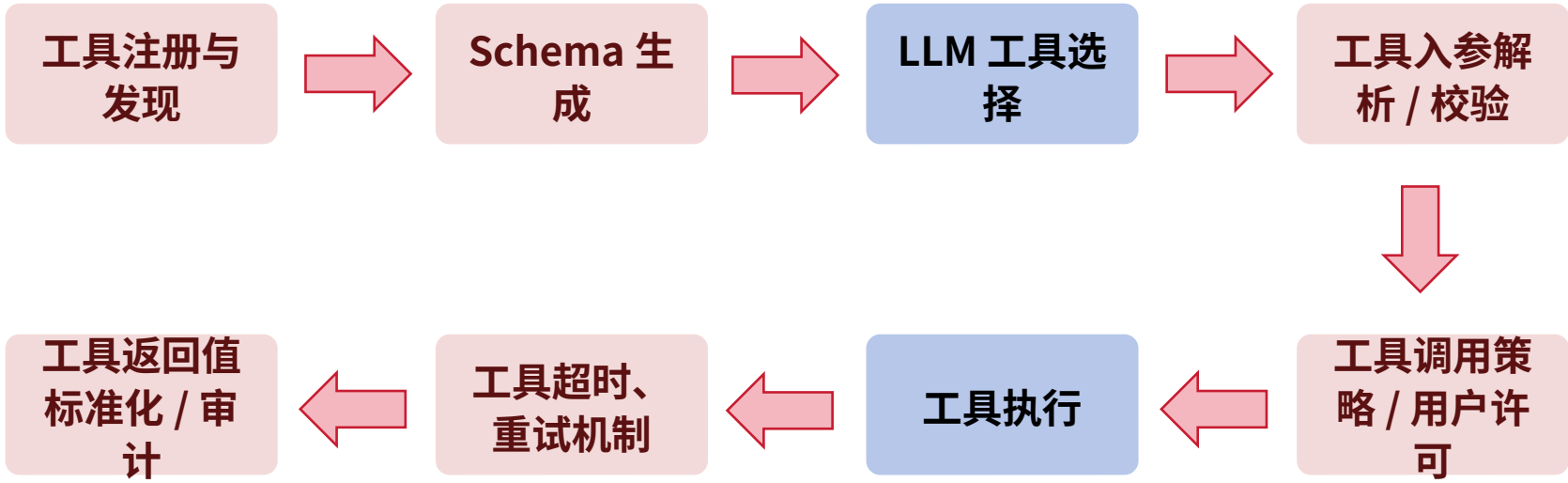
Agent 框架共性设计

工具调用：红色为 agent 框架职责

如何让 agent 发现
本地、远程、 mcp
等多种形式的工具
并统一管理

兼容不同 llm 协议协议、
不同工具协议定义

应对 LLM 生成的不确定性



将工具结果格式化为
llm 友好的格式

网络 / 服务异常
指数退避、避免重试风暴

工具调用白名单、黑名单
人工审核、小模型自动审核

Agent 框架共性设计

规划能力：所谓“规划”在不同框架中并不是同一个东西。通常有几种体现：

隐式规划

ReAct 每轮决定下一步，例如：

- LangChain Agent
- PI
- ToolCallingAgent
- Pydantic AI Agent

它实际是局部策略，不是全局计划。

周期性规划

每隔若干步重新生成计划，例如 smolagents 的 planning_interval。

显式任务账本

将目标拆成任务、状态、依赖和验收条件，例如：

- Deep Agents 的 todo/plan 能力；
- CrewAI 的 Task；
- 某些 Pydantic AI Harness capability。

静态或动态工作流图

例如：

- LangGraph
- CrewAI Flows
- AutoGen GraphFlow
- LlamaIndex Workflows
- Pydantic Graph

Agent 框架共性设计

多智能体

优势

1. **上下文隔离**：不同子任务不污染主 Agent；
2. **工具权限隔离**：不同 Agent 拥有不同 capability；
3. **并行执行**：多个独立任务 fan-out；
4. **模型异构**：不同任务使用不同模型；
5. **职责审计**：明确哪个 Agent 负责什么；
6. **外部自治实体**：跨进程、跨团队、跨组织 Agent。

代价

大量 Crew/GroupChat 示例其实只是把一次模型调用变成多次模型调用：
Planner → Researcher → Critic → Writer → Reviewer

1. Token 成倍增长；
2. 延迟增长；
3. 消息上下文重复；
4. 终止条件复杂；
5. 错误责任难以定位；
6. Agent 之间可能产生循环委托；
7. 整体质量可能被最弱 Agent 拉低。

Agent 框架概览

框架 / 产品	核心抽象	极简介绍
LangChain / LangGraph	Component / State Graph	状态化工作流
LlamaIndex	Data / Index / Retriever	数据驱动智能体
AutoGen	Agent / Message / Team	多智能体
CrewAI	Agent / Task / Crew / Flow	角色任务协作
smolagents	CodeAgent / Tool	轻量代码智能体
pi	Provider / Agent / Session	OpenClaw 的智能体运行时
Pydantic AI	Dependencies/ OutputType	类型安全智能体

后文中，主要关注 agent 框架中涉及编排和 Agent Loop 的概念

1. 不从语言开始
   Python / TS / 其他
   不讲语法细节
   不比较 API 写法
2. 只看框架抽象
   Agent 如何运行
   工具如何接入
   状态如何保存
   协作如何发生
3. 用场景判断价值
   它解决什么问题
   它适合什么场景
   它的代价是什么

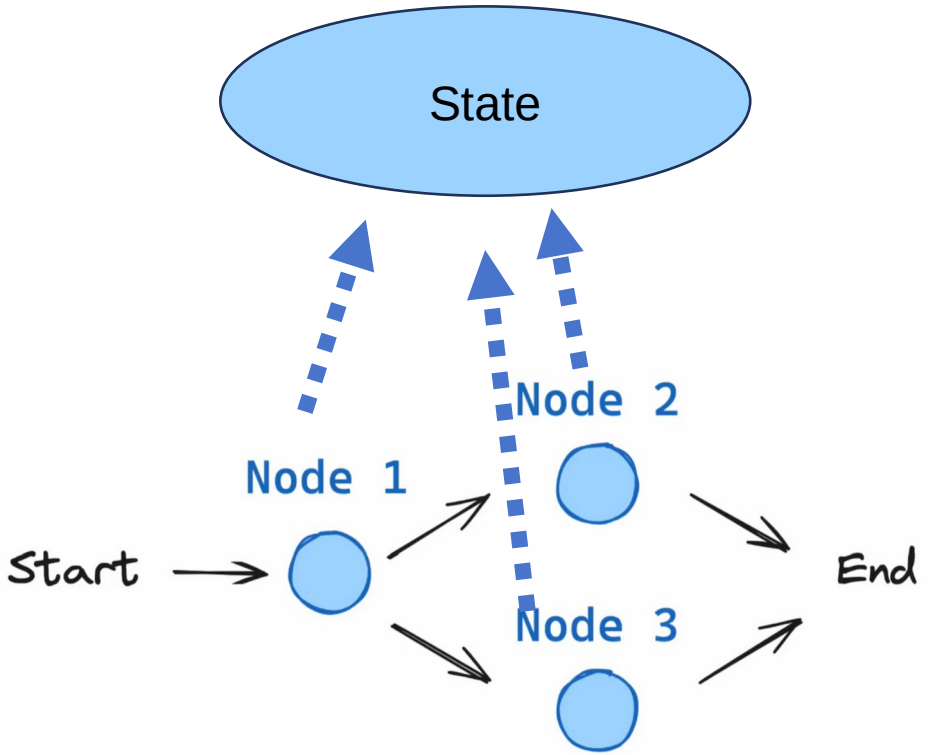
LangChain

早期 LangChain： “统一不同 LLM、 Prompt、 Retriever、 Tool 的调用接口”
核心设计： Chain （管道 / Pipeline， LCEL | 运算符）， DAG **有向无环图**，单向数据流

- 被批判高层抽象过度封装，开发者失去对 Agent 流程的控制权
- 原生不支持循环
- 状态管理隐式、无法断点续跑（ human-in-the-loop ）

现在 LangChain 1.0： 高层 API 保留，底层 Agent 运行时全部迁移到 LangGraph 状态图引擎

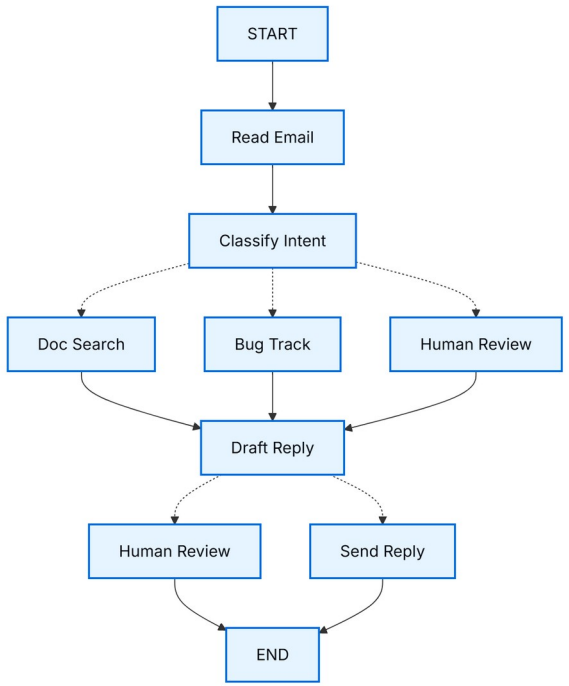




LangGragh

如何基于 langgraph 实现一个 agent

1. 定义状态
2. 定义 node
3. 编排 node 和 edge



邮件处理 agent

```python
from typing import TypedDict, Literal

# Define the structure for email classification
class EmailClassification(TypedDict):
    intent: Literal["question", "bug", "billing", "feature", "complex"]
    urgency: Literal["low", "medium", "high", "critical"]
    topic: str
    summary: str

class EmailAgentState(TypedDict):
    # Raw email data
    email_content: str
    sender_email: str
    email_id: str

    # Classification result
    classification: EmailClassification | None

    # Raw search/API results
    search_results: list[str] | None  # List of raw document chunks
    customer_history: dict | None  # Raw customer data from CRM

    # Generated content
    draft_response: str | None
    messages: list[str] | None
```

1 定义状态：邮件信息

```python
from langgraph.checkpoint.memory import MemorySaver
from langgraph.types import RetryPolicy

# Create the graph
workflow = StateGraph(EmailAgentState)

# Add nodes with appropriate error handling
workflow.add_node("read_email", read_email)
workflow.add_node("classify_intent", classify_intent)
# Add retry policy for nodes that might have transient failures
workflow.add_node(
    "search_documentation",
    search_documentation,
    retry_policy=RetryPolicy(max_attempts=3)
)
workflow.add_node("bug_tracking", bug_tracking)
workflow.add_node("draft_response", draft_response)
workflow.add_node("human_review", human_review)
workflow.add_node("send_reply", send_reply)
# Add only the essential edges
workflow.add_edge(START, "read_email")
workflow.add_edge("read_email", "classify_intent")
workflow.add_edge("send_reply", END)

# Compile with checkpointer for persistence, in case run graph with Loc
memory = MemorySaver()
app = workflow.compile(checkpointer=memory)
```

2 意图识别 node：提取并更新状态

3 写邮件 node：读取状态并写回复

LangGragh

维度	内容
核心抽象	LangChain： Model / Tool / Agent / Middleware； LangGraph： State / Node / Edge / **Checkpointer**
控制方式	Node = 执行单元； Edge = 普通边 + 条件边；**允许图有环**
运行状态	**State 全局共享状态**，所有节点读写同一份状态；用 reducer 做状态更新； checkpointer 做持久化存储（ Redis/Postgres ），支持中断恢复
记忆	State 承担短期记忆； Store 支持跨会话长期记忆
工具调用	串行 / 并行 / 动态选择 / 重试 / 错误处理；支持运行时发现
结构化输出	Provider 原生输出与 Tool Calling 自动择优，并做 Schema 校验
人工介入	敏感工具前暂停；人工批准、修改或拒绝后从状态恢复
观测与部署	LangSmith 提供 tracing 、 evaluation 、 prompt 与 deployment

Langchain 早期和当前变化对 Agent 开发者的启示

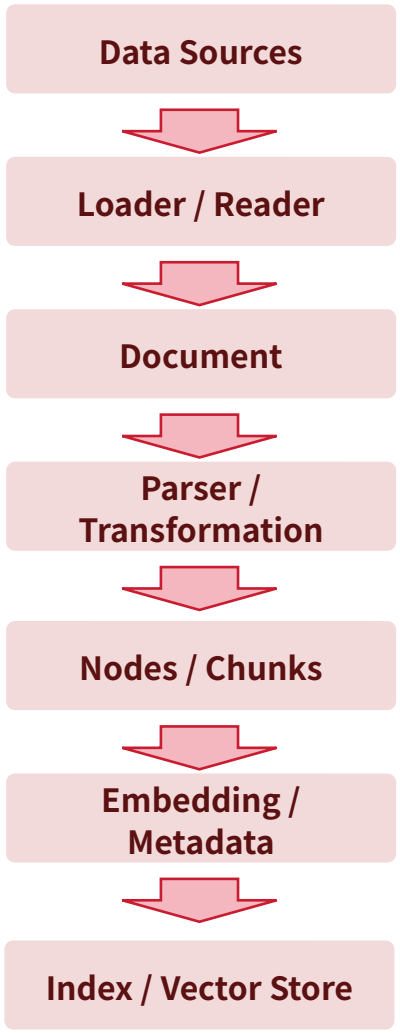
1. **原型优先 → 生产可靠性优先**
   早期目标让开发者快速写出 Demo ；后来目标支撑企业生产 Agent ，可靠性、可控性、容错、中断恢复成为第一优先级。
2. **黑盒高层魔法 → 显式可控**
   减少框架做隐式决策，把流程控制权还给开发者，框架提供基础运行原语。
3. **把 Agent 看作函数 → 把 Agent 看作状态机**
   早期 Chain 假设输入输出是纯函数；现实 Agent 是多步迭代、有记忆、会暂停、会重试的状态机。

适合 **长流程、恢复、审批、复杂路由、工具生态**

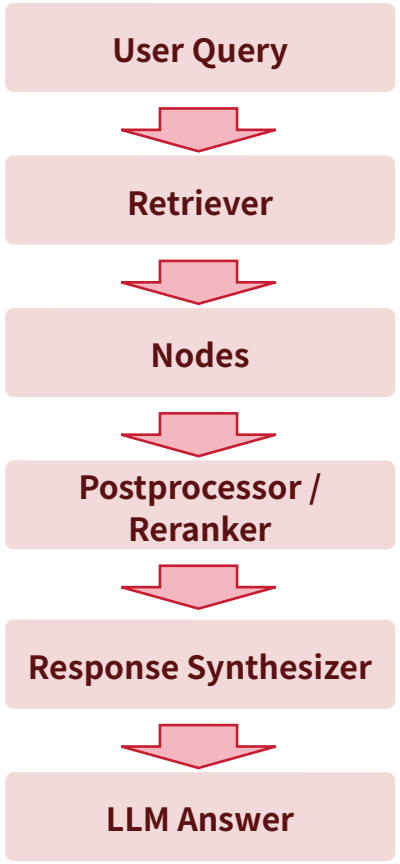
不适合 一次调用或两三个固定步骤的简单服务

LlamaIndex

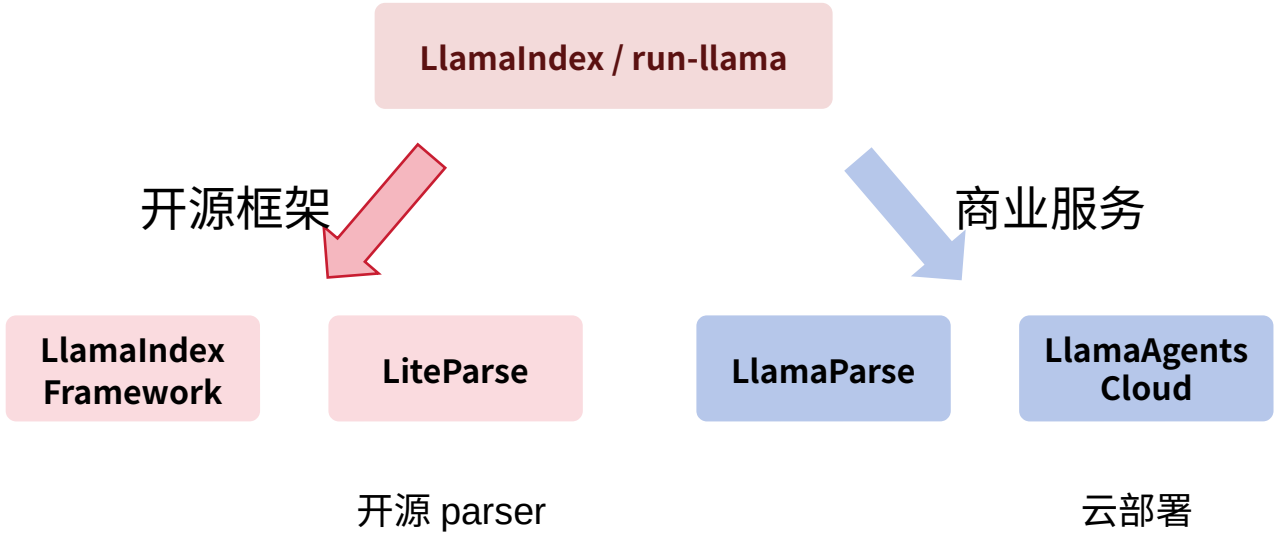
LlamaIndex 的技术基因是 RAG / data framework，但当前产品定位已经演化为“围绕企业文档和私有数据的 Agentic Application Platform”。



Indexing Plane



Query Plane



Python/TS 应用框架
原来的 RAG/Data Framework
今天被扩成了 RAG + Agent + Workflow 的通用 SDK

最成熟的数据 pipeline 商业化成 Document AI / RAG managed services

数据能力如何进入 Agent



LlamaIndex - Event 机制

“Event types describe the edges of the workflow.”

函数入参类型 = Subscribe
return Event = Publish
Event type = Edge

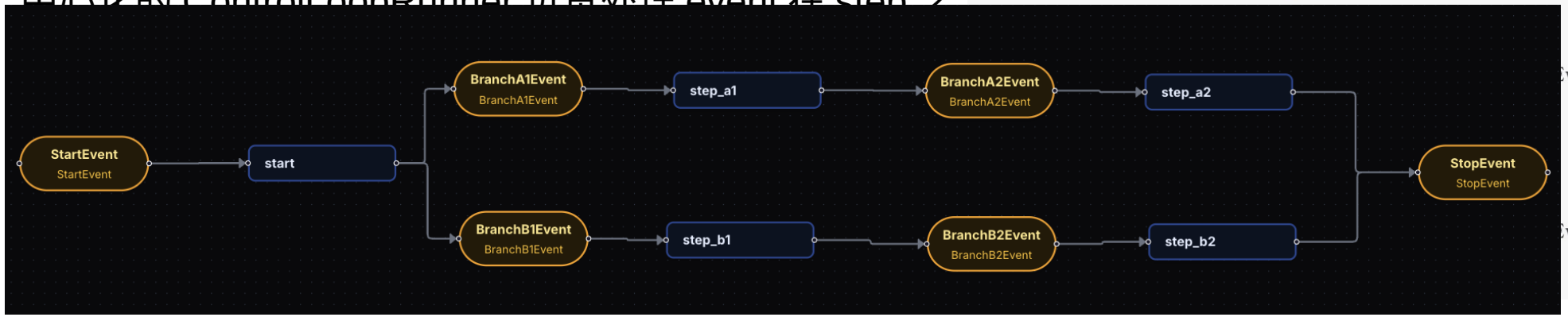
Workflow 首选 typed events 描述控制流（分支、循环）
Context 用于动态事件、共享 per-run state 和流式协调。

中心化的 ControlLoopRunner 负责处理 event 在 step 之

```python
class BranchWorkflow(Workflow):
    @step
    async def start(self, ev: StartEvent) -> BranchA1Event | BranchB1Event:
        if random.randint(0, 1) == 0:
            print("Go to branch A")
            return BranchA1Event(payload="Branch A")
        else:
            print("Go to branch B")
            return BranchB1Event(payload="Branch B")

    @step
    async def step_a1(self, ev: BranchA1Event) -> BranchA2Event:
        print(ev.payload)
        return BranchA2Event(payload=ev.payload)

    @step
    async def step_b1(self, ev: BranchB1Event) -> BranchB2Event:
        print(ev.payload)
        return BranchB2Event(payload=ev.payload)
```



LlamaIndex

维度	内容
核心抽象	Document / Node / Index / Retriever / Query Engine / Tool / Agent / Workflow
数据链路	Load → Parse / Transform → Node → Embed / Index → Retrieve → Rerank → Synthesize
Agent	FunctionAgent / ReActAgent； RAG、 Query Engine、 Workflow 都可以成为 Tool
工作流	Event + Step + Context；拓扑由 step 输入 / 输出 Event Type 推导
多智能体	AgentWorkflow； handoff 或 Agents-as-Tools
状态	Memory + Context；支持 streaming、 HITL、 checkpoint/resume
结构化输出	output_cls + Pydantic； structured_output_fn 可验证或重写
外部互操作	LlamaAgents self-host / LlamaCloud； MCP client/server； 300+ integrations

适合 **企业知识助手、复杂 RAG、研究 Agent、多检索器**

不适合 数据很少，主要难题是状态、审批与业务工作流

1. **数据与上下文是第一公民**
   LlamaIndex 的核心优势仍然来自 Document、 Node、 Retriever、 Index 等数据抽象。 Agent 和 Workflow 是建立在这套数据平面之上的能力，而不是反过来。
2. **RAG 已从应用形态变成可组合能力**
   Retriever、 Query Engine、 Document Agent 都可以被封装为 Tool，供 Agent 动态调用。它的演进方向是从“构建 RAG” 走向“让 Agent 按需获取上下文”。
3. **Workflow 采用事件驱动，而非显式 Graph**
   Step 通过输入 / 输出 Event 类型形成隐式拓扑；运行时由中心化 Event Broker / Control Loop 完成事件路由、并行调度和状态协调。灵活，但大型流程的拓扑可读性弱于显式 DAG。
4. **产品边界正在向 Document Agent Platform 扩展**
   OSS Framework 负责 RAG、 Agent 与 Workflow； LlamaParse / LlamaAgents Cloud 将文档解析、索引和 Agent 部署进一步产品化。其差异化仍集中在复杂数据和文档场景。

smolagents

Smolagents 是 huggingface 发布的 ai agent 框架。
关键思想是 JSON tool calling 并不是唯一的 Agent action language， **Python 代码可以是更强的动作表达。**

CodeAgent 与 ToolCallingAgent

维度	CodeAgent	ToolCallingAgent
动作表达	生成 Python 代码	生成 JSON 风格工具调用
组合能力	变量、循环、条件、函数；一次组合多个动作	调用结构清晰，易审计和跨模型兼容
执行要求	Python Executor；必须处理沙箱和导入权限	不执行生成代码，攻击面更小
适合任务	计算、分析、文件处理、多中间结果	API 操作、严格白名单、简单动作链

共同核心抽象

MultiStepAgent	统一多步 ReAct： thought → action → observation
Model	HF Inference 、 Transformers 、本地、 LiteLLM 、 Azure 、 Bedrock 、 MLX
Tool	名称、描述、输入类型 / 描述与输出类型
MemoryStep	记录输出、动作、观察与错误，重新写入上下文
managed_agents	管理者可把子 Agent 当成能力调用

来源： smolagents Guided Tour • ReAct • Secure Code Execution • Executors

ReAct 与安全边界

Thought → Code / Tool → **Observation**

1. **Code Action 可减少模型往返**
   一段代码完成变量保存、循环、聚合和多工具组合。
2. **表达力与安全风险同步增加**
   本地 AST 执行器并非完全安全；高风险场景使用 Docker / E2B 等隔离。
3. **轻量意味着更少生产设施**
   持久化、调度、审批、权限和长流程恢复通常由外部系统补齐。
4. **思考循环可显式约束**
   max_steps 、 planning_interval 、 final checks 与 interrupt 控制循环。

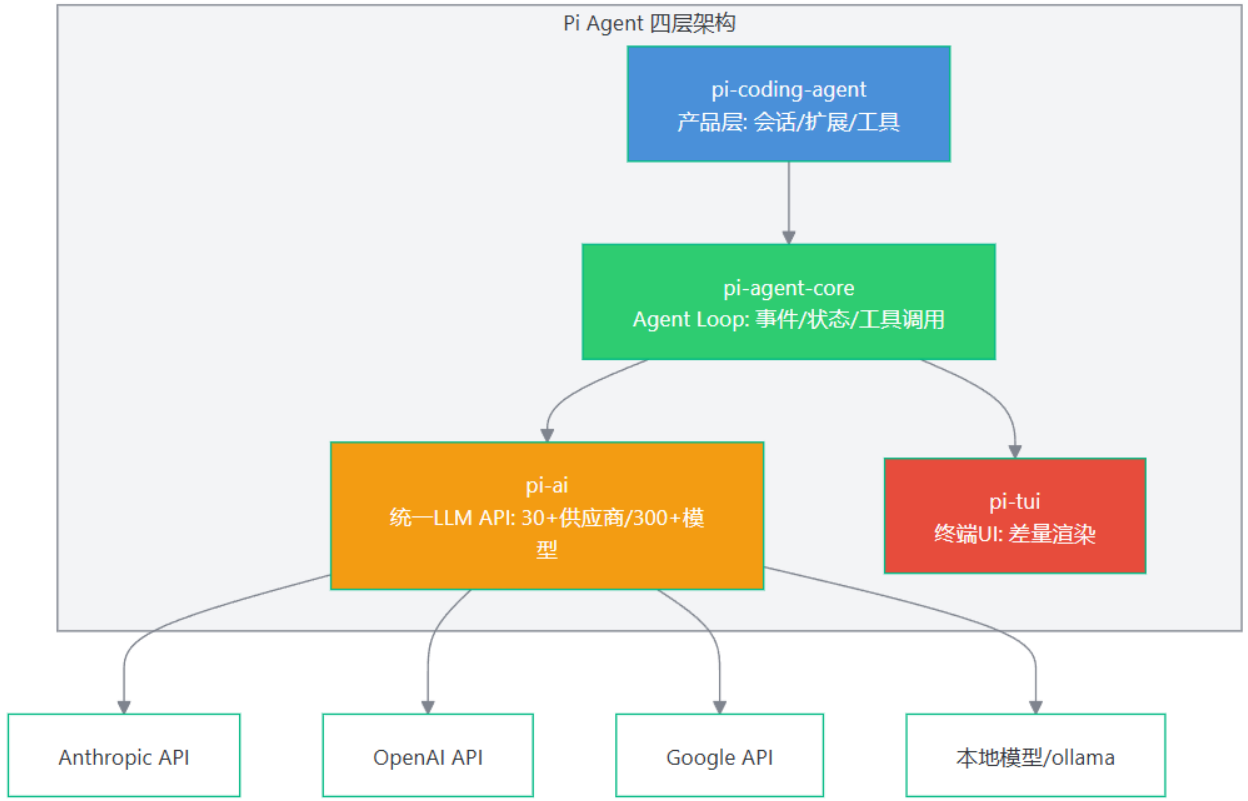
- **动作语言是框架的核心问题**
- **代码 Agent 更像动态程序生成器**
- **最佳实践是小核心 + 外部基础设施**

适合 **教学、研究、数据分析、代码驱动组合**

不适合 不可信代码直跑或企业级持久长流程

| PI

minimal agent harness： 一个最小的 agent 框架应该包含什么



核心思路

把 Agent 最小化为一个可持续运行的 Loop

- **Agent Core 只负责循环**
  接收上下文 → 调用模型 → 执行工具 → 回填结果 → 继续推理
- **模型能力与 Agent 解耦**
  pi-ai 屏蔽 Anthropic / OpenAI / Google / Local Model 差异， Agent 不绑定具体模型
- **UI 与运行时解耦**
  pi-tui 只是状态与事件的渲染层，核心 Agent 可以运行在 Terminal / Web / Server 中
- **产品能力通过 Harness 扩展**
  会话管理、工具、 Prompt 、上下文压缩等能力放到 pi-coding-agent ，而不是塞进 Agent Core

适合 **TypeScript/Node 、编码 Agent 、终端 /Web UI**

不适合 需要开箱即用的 RAG 、 Memory 、多智能体

Minimal Harness = Model Adapter + Agent Loop + Tool Runtime + Event/UI

Pydantic AI

它不以角色或图为中心，而以指令、工具、依赖类型、输出类型和模型构成泛型容器，降低 Agent 接入 Python 业务系统的接口风险。

核心抽象	作用
Agent[DepsT, OutputT]	在类型层同时声明运行依赖与最终输出
RunContext[DepsT]	工具、动态指令和验证器读取本次依赖与上下文
deps_type	数据库、 HTTP Client 、用户、配置等外部服务类型
output_type	标量、容器、 TypedDict 、 dataclass 、 Model 、 Union 、函数
Tool / Toolset	从函数签名生成 Schema ，并执行模型工具调用
Output Validator	同步 / 异步业务校验； ModelRetry 反馈模型重试
Model / Provider	统一多模型调用，并可在运行时指定或切换
Capabilities	组合工具、 hooks 、 instructions 与 model settings

- **真正用户是业务代码，而不是流程图**
- **Schema + 外部校验 + 有限重试 + 评测**
- **与 FastAPI 的依赖和返回值心智模型相近**

来源： Pydantic AI Agent · Dependencies · Output · Durable Execution · Evals · Logfire

类型契约如何穿过 Agent

业务依赖 → Context → Agent / Tools → Output → **Validator**

六项工程机制

1. **依赖注入摆脱全局变量**
   deps 以强类型传入，工具更易测试、替换与部署。
2. **结构化输出不是解析 JSON**
   output_type 生成 Schema ， Pydantic 再验证并保留类型。
3. **验证失败进入自纠错**
   ModelRetry 将业务错误反馈模型；重试预算限制循环。
4. **类型安全不等于事实安全**
   事实、权限和副作用仍需业务校验与 guardrail 。
5. **Durable Execution 借力成熟引擎**
   Temporal 、 DBOS 、 Prefect 、 Restate 承担可靠长流程。
6. **观测与评测是一条链**
   Logfire + OpenTelemetry 追踪； Evals 可检查输出与工具轨迹。

适合 Python 后端、结构化对象、依赖注入、验证与 OTel

不适合 角色团队 UI 、极简 Demo 、自绘复杂状态图

AutoGen

微软推出的面向多智能体的 Agent 框架。

AutoGen 的四个关键设计判断

- **Agent 与 Runtime 分层**
  AgentChat 面向任务编排， Core 面向 Actor/Message Runtime 。
- **Termination 是一等抽象**
  自主性越高，越需要显式定义停止条件与资源边界。
- **Orchestration 独立于 Agent**
  同一组 Agent 可以采用 RoundRobin / Selector / Swarm / Graph 等不同协议。
- **State 是系统设计问题，而非 Prompt 问题**
  Agent 、 Team 、 Context 、 Memory 均需显式生命周期管理。

适合 **学习多智能体思想**

不适合 需要长期官方主线支持的新生产项目

框架四层

层级	核心抽象	作用
Core	Agent / AgentId / Message / Runtime	事件驱动、可扩展消息运行时
AgentChat	Agent / Message / Team / Termination	高层任务驱动 API
Extensions	Model / Tool / Workbench / Executor	模型、 MCP 、 Docker 、 gRPC
Studio	可视化原型工具	低代码搭建与观察团队

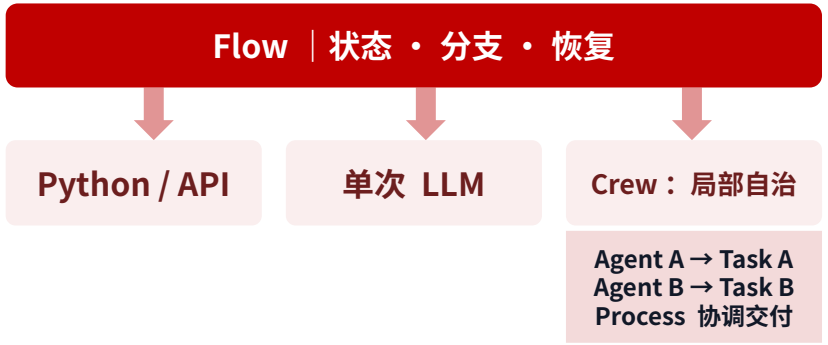
模式	谁决定下一步	核心机制
AssistantAgent （单智能体）	自己	LLM + Tools
RoundRobin	固定顺序	轮询
SelectorGroupChat	中央 Selector	LLM 动态选 Agent
Swarm	当前 Agent	Handoff
GraphFlow	开发者	显式图
Magentic-One	Orchestrator	Planning + Ledger + Replanning

Crew AI

Agent 有角色、目标和工具， Task 定义交付物， Crew 组织协作； Flow 用事件、状态与路由为业务流程提供确定性边界。

核心抽象	关键内容
Agent	role 、 goal 、 backstory 、 tools 、 LLM 、是否允许委派
Task	description 、 expected_output 、负责人、上下文依赖、输出模型
Crew	Agent 与 Task 的顶层组织， kickoff 并负责最终交付
Process	sequential 、 hierarchical 等任务组织模式
Flow	start / listen / router 、状态、分支、循环、持久化与恢复
Guardrail	校验 Task 输出；不通过时反馈并重试
Memory / Knowledge	提供历史经验或领域知识；检索质量与权限需单独评估

Flow 包住 Crew



关键机制

1. **角色设定不等于能力隔离**
   真正边界来自工具、数据、模型和执行环境。
2. **expected_output 是交付接口**
   结合 JSON / Pydantic ，让下游 Task 稳定消费。
3. **Crews 与 Flows 解决不同问题**
   Flow 负责可审计主流程， Crew 只承接开放推理。
4. **高层抽象可能掩盖成本**
   用 tracing 、任务指标和终止条件衡量收益。

适合 **角色协作、内容生产、业务流程原型**

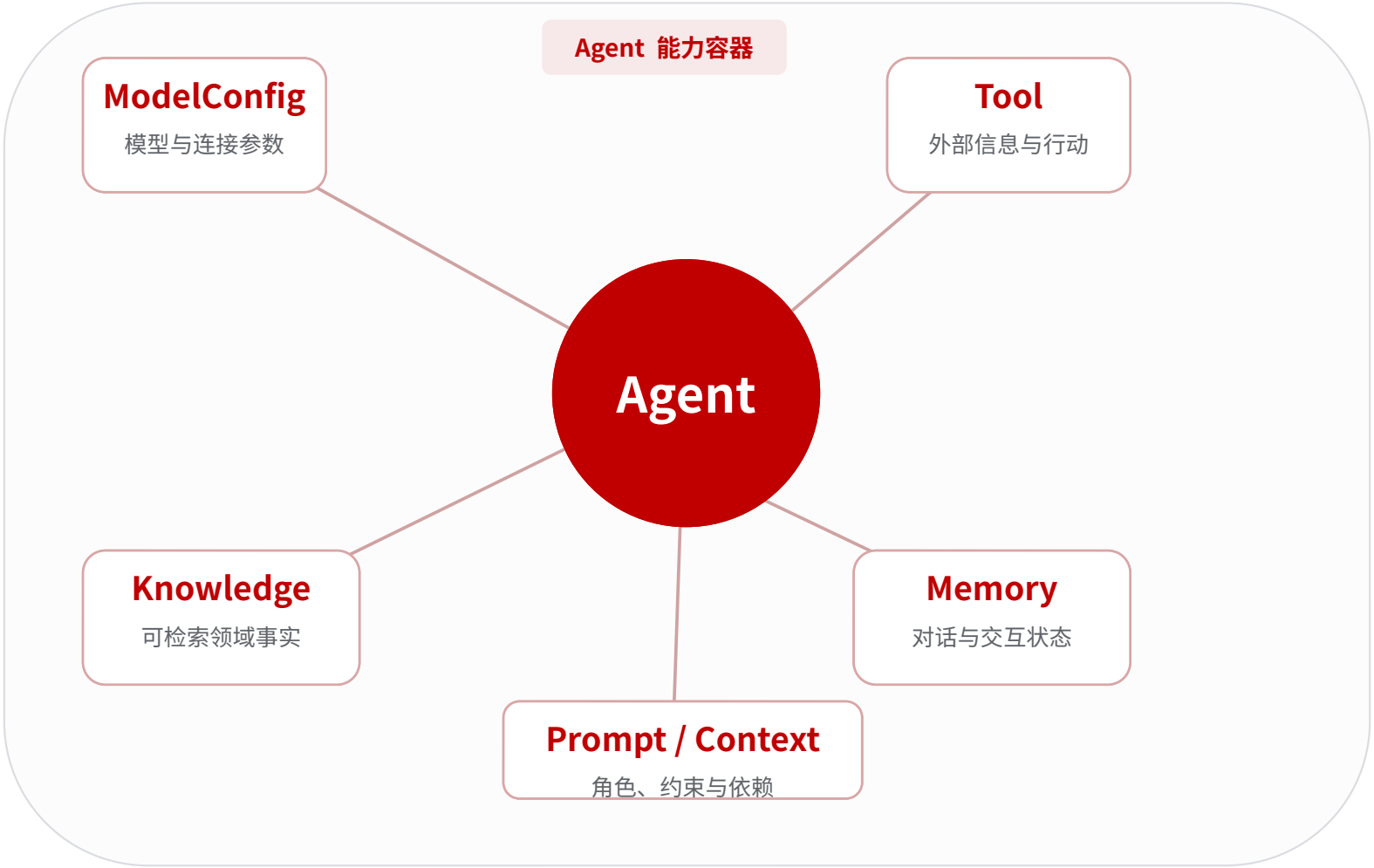
不适合 极低延迟、单工具循环、精细低层状态机

框架选择

Agent 框架的分化，本质是控制权放在哪里：状态图、数据检索、消息协作、角色任务、代码动作、会话运行时，还是类型边界。

你的主问题	优先研究	原因
长流程、恢复、审批、复杂路由	LangGraph	状态图、 checkpoint 、 HITL 、 time travel 是中心能力
私有数据、复杂 RAG 、多检索器	LlamaIndex	数据、 Index 、 Retriever 、 Query Engine 与 Agent 原生衔接
理解或维护消息式多 Agent	AutoGen	Team 与消息运行时成熟；新项目必须考虑维护状态
角色分工、任务协作、业务易理解	CrewAI	Agent / Task / Crew 组织隐喻直接； Flow 补确定性
极简 Agent 、代码动作、教学实验	smolagents	抽象少； CodeAgent 的动作语言特征鲜明
TS 编码 Agent 、会话分支、嵌入 UI	pi	Provider / Agent / Session 分层；树状会话突出
Python 类型安全、结构化业务接口	Pydantic AI	依赖与输出类型、验证和重试是一等能力

TongAgents SDK：从 Agent 到 Workflow



复杂度升级路径

01 ModelConfig

统一 OpenAI-compatible 模型入口

02 单个 Agent

推理、工具、知识与记忆协同

03 Workflow

分支、并行、调试与人工接管

原则：先把能力收进 Agent，再把业务结构交给 Workflow

TongAgents SDK：安装与运行环境

安装不是一步到位：按能力选择 extras，并先固定 Python 与模型环境。

安装目标	命令 / 配置	何时使用
基础运行	pip install tongagents	只安装核心库；支持 Python 3.11 或 3.12
运行官方示例	tongagents[examples]	补齐示例代码依赖；适合课堂与快速验证
本地知识库	tongagents[knowledge]	向量数据库、文本嵌入、文档加载与切分
文本检测	tongagents[detection]	Surya 检测模型及 PyTorch 、 Transformers 等依赖
组合能力	[knowledge,examples,detection]	一次安装多个可选组；避免无关重量依赖
模型环境	3 个环境变量	MODEL_NAME / MODEL_URL / MODEL_API_KEY ；不要把密钥写进代码
兼容入口	ModelConfig	OpenAI-compatible 模型可通过统一配置接入

TongAgents SDK：第一个 ReactAgent

```python
import os
from tongagents.agents.llm.base import ModelConfig
from tongagents.agents.llm_agent import (
    LLMRunContext, ReactAgent, ReactAgentSetting
)

settings = ReactAgentSetting(
    llm_config=ModelConfig(
        model_name=os.environ["MODEL_NAME"],
        url=os.environ["MODEL_URL"],
        api_key=os.environ["MODEL_API_KEY"],
    )
)
agent = ReactAgent(
    dep_context=LLMRunContext(),
    agent_settings=settings,
)
result = agent.step(" 请输出  Hello World")
print("".join(m.content for m in result))
```

四个关键对象

01	**ModelConfig**	模型名、URL、密钥与生成参数
02	**ReactAgentSetting**	Agent 行为与 LLM 配置入口
03	**LLMRunContext**	本次运行依赖与上下文
04	**ReactAgent**	ReAct 主体；可挂 Tool 与 Knowledge

运行接口怎么选?

step	stream	run
完整结果 / astep	流式迭代 / astream	长会话 / arun

Agent 类型与运行接口

Agent 类型决定行为边界；运行接口决定一次执行如何返回。

开箱即用 Agent	主要能力 / 适用边界
ReactAgent	ReAct；工具调用与知识库集成
TypedReactAgent	将模型输出解析为指定 Pydantic BaseModel
TongLLMAgent	加载平台角色、开场白等配置；继承 ReactAgent
PlanAgent	可插拔 Planner；任务规划与执行
自定义 Agent	继承 Agent / WorkflowAgent，复用统一运行协议

- **默认从 ReactAgent 开始，先证明单体 Agent 能力**
- **需要强结构输出时切换 TypedReactAgent / FormatLLMAgent**

运行接口如何穿过 Agent

Event → step / stream → WF → END → Result

六项运行语义

1. **step：完整响应**
   运行到 End 节点后返回完整结果，适合结构化输出。
2. **stream：单次流式**
   返回迭代器；可逐块消费 LLM 与节点输出。
3. **run：长会话**
   输入与输出都是迭代器，适合语音等双向流。
4. **异步接口成对提供**
   astep / astream / arun 对应同步接口。
5. **Agent 本身按 Workflow 执行**
   数据在内部流动，直到 End 节点形成输出。
6. **复杂场景升级 Workflow**
   多节点、分支、并行与恢复交给显式编排。

选型 **按输出契约、交互形态和是否需要规划来选择**

原则 接口保持一致，复杂度逐层上移

Tool：从 Schema 到治理

```python
from tongagents.tools.tool_manager import tool

@tool(
    name="weather_query",
     description=" 查询指定地点天气 ",
    retry_times=3,
)
def weather_tool(location: str, time: str):
    return f"{location} 在 {time} 天气晴朗 "

agent = ReactAgent(
    dep_context=LLMRunContext(),
    agent_settings=settings,
    tools=[weather_tool],
)
agent.step(" 今天北京天气怎么样？ ")
```

一次工具调用如何发生

1. **生成契约** — 函数签名 + docstring → Schema
2. **模型选工具** — name / description 进入模型上下文
3. **参数校验** — 基础类型 / Pydantic / dataclass
4. **执行与治理** — pre_hook / auth / retry_times
5. **回写 Observation** — post_hook → LLM 组织最终回答

Tool 形态：本地 @tool | 远程 ToolService | MCP | ToolHook 治理

TongAgents SDK：三种 Tool 接入路径

PYTHON

```python
from tongagents.tools.tool_service import ToolService
from tongagents.tools.mcp_client import MCPClient
from tongagents.tools.mcp_tool_manager import
MCPToolManager

remote = ToolService(REMOTE_URI).get_tool("web_search")

client = MCPClient("weather_server.py", timeout=10)
mcp_tools =
MCPToolManager.initialize_from_mcp(client)

agent = ReactAgent(
   dep_context=LLMRunContext(),
   agent_settings=settings,
    tools=[weather_tool, remote, *mcp_tools],
)
result = agent.step(" 查询天气并搜索出行建议 ")
```

接入与治理路径

01	**@tool**	进程内函数；签名自动生成 Schema
02	**ToolService**	按唯一名称获取远程工具服务
03	**MCP**	标准化连接数据源、工具与资源
04	**ToolHook**	pre / post / auth 将治理与逻辑解耦

怎么选接入方式?

LOCAL
低延迟 / 同进程

REMOTE
服务复用 / 统一网关

MCP
开放协议 / 多工具

Knowledge 与 Memory：事实检索和交互状态不能混为一谈

KNOWLEDGE 知识库

回答	“制度、文档或资料里如何规定？”
来源	文档、数据库、业务资料
单位	Document / SearchResult
实现	Chroma / Milvus / ElasticSearch

MEMORY 记忆

回答	“我刚才选择了什么、做到哪一步？”
来源	对话与 Agent 交互过程
单位	Message / MessageChunk
实现	SimpleMemory / PersistenceMemory

KnowledgeStore 数据路径

Loader → Splitter → Embedding → Vector Store → search(query)

Message 最小结构

role　chunks　metadata

Knowledge：统一检索接口

```python
from tongagents.knowledge.vectorstore.chroma import
ChromaVectorStore
from tongagents.tools.loader.directory_loader import
DirectoryLoader
from tongagents.tools.splitter.impl import default_text_splitter

documents = DirectoryLoader(
    input_files=["knowledge_store.txt"]
).load()
chunks = default_text_splitter.split_document(documents)

store = ChromaVectorStore()
store.add_documents(chunks)

agent = ReactAgent(
    dep_context=LLMRunContext(),
    agent_settings=settings,
    tools=[store],
)
agent.step(" 从知识库查询皮卡丘的身高 ")
```

四个统一抽象

01	**Document**	文档内容与元数据的交换单位
02	**Loader / Splitter**	负责读取、清洗与切分，而非检索
03	**KnowledgeStore**	add_documents / search / delete / clear
04	**SearchResult**	把不同引擎的召回结果统一交给 Agent

默认实现怎么选?

- **Chroma** — 本地起步 / 课堂实验
- **Milvus** — 大规模向量检索
- **ElasticSearch** — 全文 / 混合检索

Workflow：复杂度升级工具

当单个 Agent 难以承载固定业务结构，再把分支、并行、恢复和人工确认显式化。

检索 → 判断 → 执行 → 生成



三种构建方式

add_node + add_edge

快速构建；适合小型流程

@node_declare

继承与复用；代码即结构

WorkflowConfig

配置化、发布与持久化

升级到 Workflow 的信号

- ✓ 多个上游结果必须汇聚
- ✓ 条件判断、分支或循环
- ✓ 需要 debug 事件观测
- ✓ 需要 interrupt / resume 人工接管

Workflow：节点 + 边 + Event

```python
from tongagents.workflow.simple_workflow import (
    START, END, Workflow
)

workflow = Workflow()
workflow.add_node(input_node)
workflow.add_node(llm_node)

workflow.add_edge(START, input_node.config.name)
workflow.add_edge(
    input_node.config.name,
    llm_node.config.name,
)
workflow.add_edge(llm_node.config.name, END)

result = workflow.step(" 你好 ")
print(result)
```

四个核心对象

01	**Workflow**	持有节点、边与运行上下文
02	**Node**	function / NodeBase / Agent / 子 Workflow
03	**Edge**	规定数据从哪个节点流向哪里
04	**Event**	节点之间统一的数据与调度单位

构建方式怎么选?

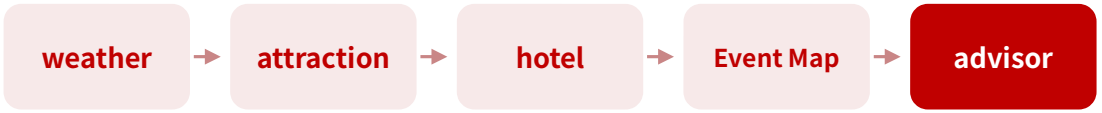
- **add_node** 快速试验
- **@node_declare** 继承复用
- **WorkflowConfig** 持久化配置

Workflow 运行时：汇聚、调试、人工接管

EVENT 汇聚

默认	任一上游事件到达，节点即可被触发
汇聚	等待全部上游就绪，再形成完整上下文
配置	MERGE_EVENTS_AS_EVENT_MAP
适用	并行检索、多源决策、缺一不可的数据

并行数据 → 完整上下文



DEBUG / HUMAN IN THE LOOP

调试	node_start / node_end
异常	node_process / node_exception
中断	interrupt() 暂停并等待人类输入
恢复	ResumeCommand 恢复；支持多次中断与幂等

调试事件生命周期



课后习题

作业目标

在现有 cli-sample 基础上增加真实流式输出能力，让模型生成一个 chunk，终端就立即显示一个 chunk，降低首字等待时间。

核心任务

在 ModelConfig 中设置 stream=True，开启模型端 Streaming

识别 STREAM_START / STREAM_CONTENT / STREAM_FINISH 等流事件

为现有 CLI Agent 增加 stream() 方法，只提取并返回正文 chunk

修改 REPL，通过 end="" 和 flush=True 实时打印内容

验证普通问答、文件工具调用和退出逻辑在 Streaming 模式下仍然正常

关键要求

必须使用 TongAgents 提供的真实流式接口，不能拿到完整答案后再模拟逐字打印；只显示非空的 STREAM_CONTENT.content，不输出 reasoning_content；完整消费事件流，并避免回答结束后重复打印完整内容。

最终效果

CLI 的回答方式从：

用户输入 → 等待完整答案 → 一次性显示

升级为：

用户输入 → 模型持续生成 chunk → 终端实时显示 → 回答自然结束